Question 1: Coordinate Systems

Questions from the assignment

1. Write an R program to visualize the dataset using a Cartesian coordinate system (bar chart).
2. Write another R program (using coord_polar()) to visualize the same data on a 24-hour clock-style Polar coordinate system.
3. Compare the effectiveness of both visualizations.
4. Explain which coordinate system is more appropriate for cyclical/time-of-day data.
5. Justify your answer based on data interpretation.

Aim

To compare store footfall across the supplied times using a conventional bar chart and a circular chart that communicates the repeating 24-hour cycle.

Procedure

Enter the eight time/footfall pairs in data frames. Call ggplot2 functions with the ggplot2:: prefix, so no library() call is needed. Use geom_col() for the Cartesian chart and coord_polar() for the clock-style chart. ggplot2 must still be installed.

Program

```
footfall <- data.frame(
  hour = c("6 AM", "9 AM", "12 PM", "3 PM",
           "6 PM", "9 PM", "12 AM", "3 AM"),
  visitors = c(10, 45, 80, 60, 95, 70, 20, 5)
)

cartesian <- ggplot2::ggplot(footfall,
                             ggplot2::aes(x = hour, y = visitors)) +
  ggplot2::geom_col(fill = "steelblue") +
  ggplot2::labs(title = "Store Footfall by Time",
                x = "Hour of day", y = "Store footfall") +
  ggplot2::theme_minimal()
print(cartesian)

clock_order <- c("12 AM", "3 AM", "6 AM", "9 AM",
                 "12 PM", "3 PM", "6 PM", "9 PM")
clock_data <- data.frame(
  hour = factor(clock_order, levels = clock_order),
  visitors = c(20, 5, 10, 45, 80, 60, 95, 70)
)
polar <- ggplot2::ggplot(clock_data,
```

```
                    ggplot2::aes(x = hour, y = visitors)) +
  ggplot2::geom_col(fill = "darkorange") +
  ggplot2::coord_polar(start = 0, direction = 1) +
  ggplot2::labs(title = "Store Footfall on a 24-hour Clock",
                x = "Hour of day", y = "Store footfall") +
  ggplot2::theme_minimal()
print(polar)
```

Input :

RStudio-style practice mockup | Coordinate systems

```
footfall <- data.frame(
  hour = c("6 AM", "9 AM", "12 PM", "3 PM",
           "6 PM", "9 PM", "12 AM", "3 AM"),
  visitors = c(10, 45, 80, 60, 95, 70, 20, 5)
)
cartesian <- ggplot2::ggplot(footfall,
  ggplot2::aes(hour, visitors)) +
  ggplot2::geom_col(fill = "steelblue")
clock_order <- c("12 AM", "3 AM", "6 AM", "9 AM",
  "12 PM", "3 PM", "6 PM", "9 PM")
clock_data <- data.frame(hour = factor(clock_order),
  visitors = c(20, 5, 10, 45, 80, 60, 95, 70))
polar <- ggplot2::ggplot(clock_data,
  ggplot2::aes(hour, visitors)) + ggplot2::coord_polar()
```

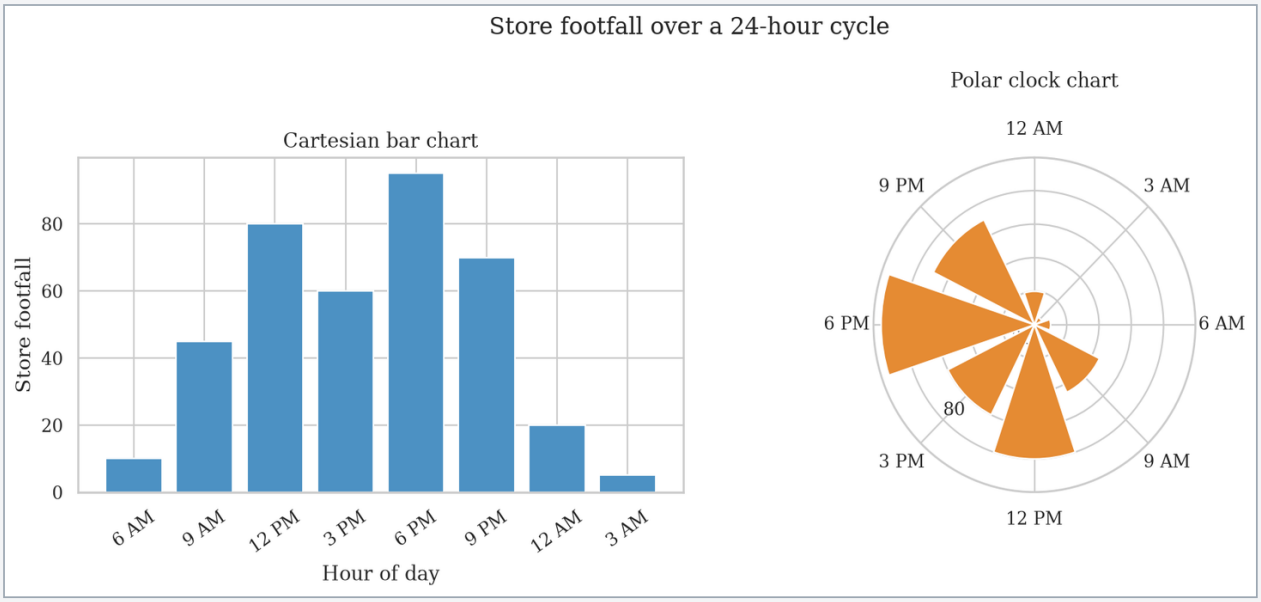
Console / sample input (illustrative)

```
> Data and code are shown for illustration; this is not a live software capture.

> See the Program section for the complete script.
```

Output :

RStudio-style practice mockup | Plots: coordinate-system comparison



Illustrative output from supplied data; not an actual RStudio screenshot.

Result

1. The ggplot2::ggplot() call creates a Cartesian bar chart with hour labels on the x-axis and footfall on the y-axis.

2. The second ggplot2 plot uses ggplot2::coord_polar() with the hours ordered from 12 AM through 9 PM, placing observations around a 24-hour cycle.

3. The Cartesian chart makes bar heights and exact differences easier to compare. The polar chart makes the daily cycle and its wraparound more visible, though radial lengths are less direct to compare.

4. A polar layout is more appropriate for showing the cyclic structure of time-of-day values; the Cartesian chart remains better when precise magnitude comparison is the priority.

5. Footfall peaks at 6 PM (95) and is lowest at 3 AM (5). Showing the hours in a circle reflects that 9 PM, midnight, and early morning belong to one continuing daily cycle.

Question 2: Correlation Heatmap

Questions from the assignment

1. Write a Python program (using pandas.corr() and seaborn.heatmap()) to compute and display the correlation matrix of Attendance, Study Hours, Sleep Hours, and Marks.
2. Identify the pair of variables with the strongest correlation.
3. Explain what a diverging color scale communicates in a correlation heatmap.
4. Discuss one limitation of using correlation to interpret this data.
5. Suggest one additional variable that could improve this analysis.

Aim

To calculate Pearson correlations among attendance, study hours, sleep hours, and marks, then present the relationships in an annotated heatmap.

Procedure

Enter the six student records into a pandas DataFrame, select its numeric columns with corr(), and pass the result to seaborn.heatmap(). The diverging palette is centered at zero and fixed to the correlation range from -1 to 1. Install pandas, seaborn, and matplotlib before running the script.

Program

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.DataFrame({
    "Attendance": [92, 75, 88, 60, 95, 70],
    "Study Hours": [3.5, 1.5, 2.8, 1.0, 4.0, 2.0],
    "Sleep Hours": [6.5, 8.0, 7.0, 5.5, 7.5, 6.0],
    "Marks": [88, 62, 80, 48, 91, 58]
})
correlation = data.corr()
sns.heatmap(correlation, annot=True, fmt=".2f",
            cmap="coolwarm", center=0, vmin=-1, vmax=1)
plt.title("Student Data Correlation Heatmap")
plt.tight_layout()
plt.show()
```

Input :

RStudio-style practice mockup | Correlation heatmap (Python)

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
data = pd.DataFrame({
    "Attendance": [92, 75, 88, 60, 95, 70],
    "Study Hours": [3.5, 1.5, 2.8, 1.0, 4.0, 2.0],
    "Sleep Hours": [6.5, 8.0, 7.0, 5.5, 7.5, 6.0],
    "Marks": [88, 62, 80, 48, 91, 58]
})
correlation = data.corr()
sns.heatmap(correlation, annot=True, cmap="coolwarm", center=0)
plt.show()
```

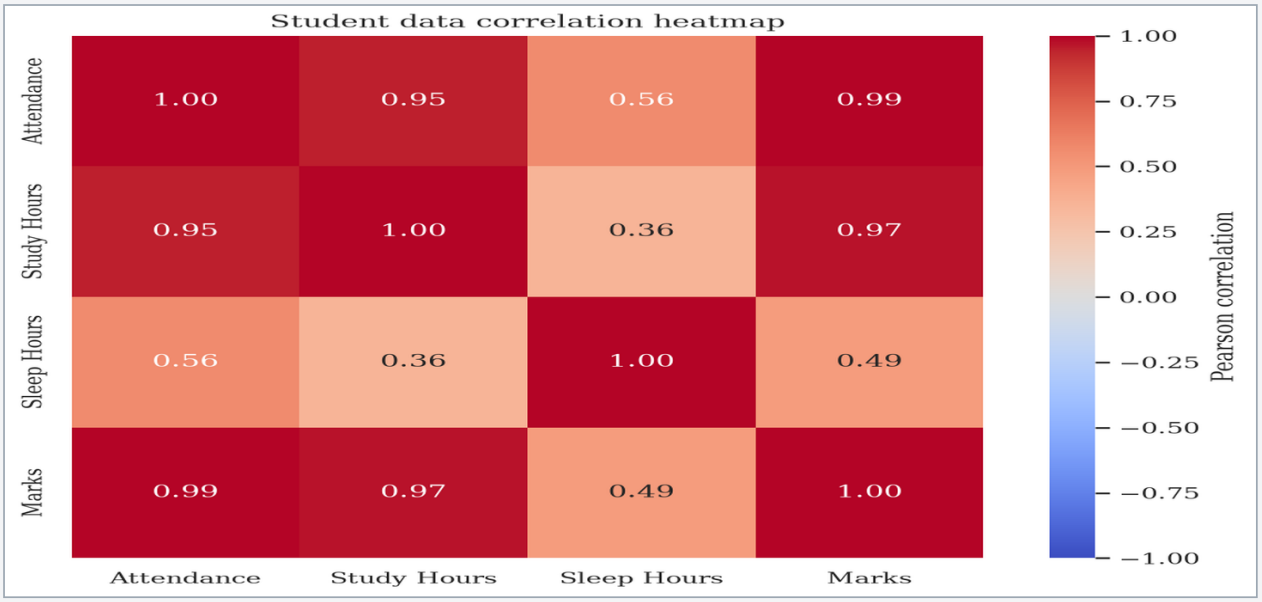
Console / sample input (illustrative)

> Data and code are shown for illustration; this is not a live software capture.

> See the Program section for the complete script.

Output :

RStudio-style practice mockup | Plots: correlation heatmap



Illustrative output from supplied data; not an actual RStudio screenshot.

Result

1. The Python program computes the correlation table with data.corr() and displays the coefficients with seaborn.heatmap().

2. Attendance and marks have the strongest off-diagonal correlation in this dataset (Pearson r is approximately 0.994). Study hours and marks are also strongly correlated (approximately 0.969). These estimates use only six students.

3. A diverging palette assigns different hues to negative and positive correlations, with a neutral midpoint near zero. The color intensity indicates how close a coefficient is to -1 or +1.

4. Correlation does not show cause and effect. The sample contains only six students, so one or two observations can strongly affect the coefficients.

5. Previous-term GPA would provide a useful measure of prior academic performance and help put current marks in context.

Question 3: Faceting / Small Multiples

Questions from the assignment

1. Write an R program (using facet_wrap()) to create a faceted plot showing the average-score trend for each Subject separately.
2. Choose an appropriate variable to facet by and justify the choice.
3. Compare trends across the facets.
4. Explain the advantage of faceting over a single combined plot for this dataset.
5. Suggest a scenario where faceting would not be suitable.

Aim

To compare average-score changes across three semesters while giving each subject its own small plot.

Procedure

Enter the twelve subject-semester scores explicitly in a data frame, then order the Semester factor. Use ggplot2::ggplot(), ggplot2::geom_line(), and ggplot2::facet_wrap() with namespace prefixes; no package attachment or repeated-value helper is needed. ggplot2 must be installed.

Program

```
scores <- data.frame(
  Subject = c("Maths", "Maths", "Maths",
              "Physics", "Physics", "Physics",
              "English", "English", "English", "CS", "CS", "CS"),
  Semester = c("Sem1", "Sem2", "Sem3", "Sem1", "Sem2", "Sem3",
               "Sem1", "Sem2", "Sem3", "Sem1", "Sem2", "Sem3"),
  Score = c(72, 78, 81, 65, 70, 75,
            80, 82, 85, 74, 83, 90)
)
scores$Semester <- factor(scores$Semester,
                          levels = c("Sem1", "Sem2", "Sem3"))

ggplot2::ggplot(scores,
                ggplot2::aes(x = Semester, y = Score, group = 1)) +
  ggplot2::geom_line(color = "steelblue") +
  ggplot2::geom_point(color = "darkorange", size = 2) +
  ggplot2::facet_wrap(~ Subject, ncol = 2) +
  ggplot2::labs(title = "Average Score by Subject and Semester",
                x = "Semester", y = "Average score") +
  ggplot2::theme_minimal()
```

Input :

RStudio-style practice mockup | Faceting / small multiples

```
scores <- data.frame(
  Subject = c("Maths", "Maths", "Maths",
    "Physics", "Physics", "Physics",
    "English", "English", "English", "CS", "CS", "CS"),
  Semester = c("Sem1", "Sem2", "Sem3", "Sem1",
    "Sem2", "Sem3", "Sem1", "Sem2", "Sem3", "Sem1", "Sem2", "Sem3"),
  Score = c(72, 78, 81, 65, 70, 75,
    80, 82, 85, 74, 83, 90)
)
ggplot2::ggplot(scores, ggplot2::aes(Semester, Score)) +
  ggplot2::geom_line() + ggplot2::facet_wrap(~ Subject)
```

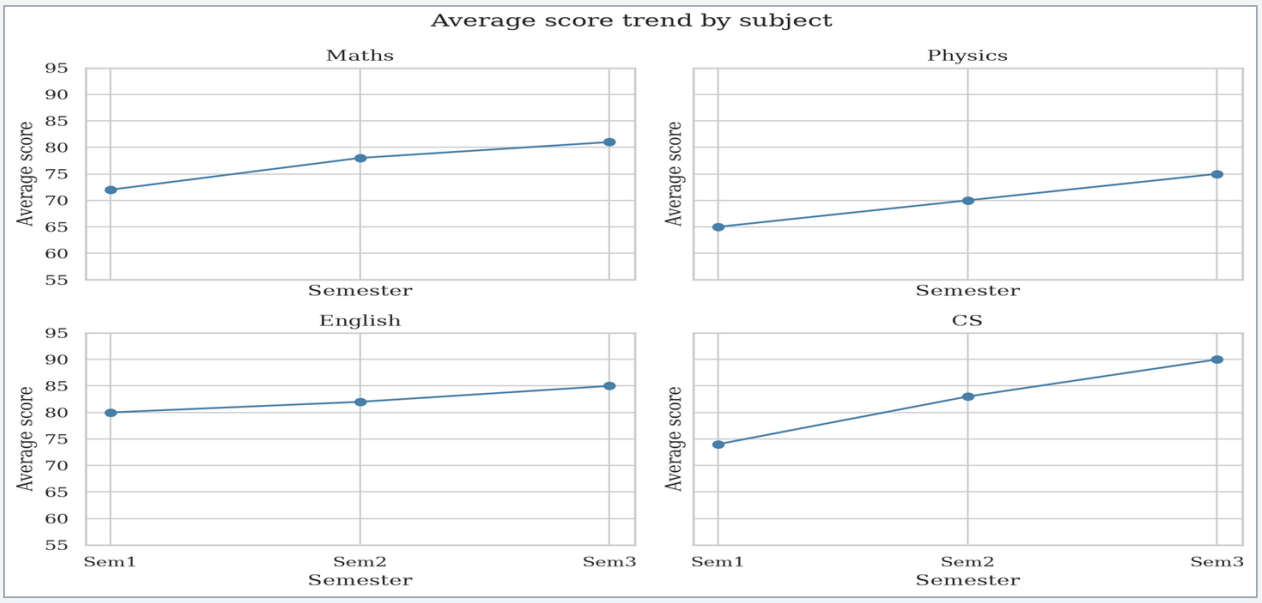
Console / sample input (illustrative)

> Data and code are shown for illustration; this is not a live software capture.

> See the Program section for the complete script.

Output:

RStudio-style practice mockup | Plots: faceted semester trends



Illustrative output from supplied data; not an actual RStudio screenshot.

.

Result

1. The R program uses facet_wrap(~ Subject) to draw a separate semester trend for each subject.

2. Subject is the appropriate faceting variable because the task is to compare semester progression separately for Maths, Physics, English, and CS. Semester remains the shared x-axis.

3. Every subject improves from Sem1 to Sem3. CS rises the most (74 to 90, +16); Physics rises by 10, Maths by 9, and English by 5 points.

4. Separate panels reduce line overlap and make each subject's direction easy to read. Shared axes still support comparison of score levels.

5. Faceting is less suitable when there are hundreds of subjects or categories: panels become too small to read. A summary or interactive filter would work better in that case.

Question 4: Stacked Bar Charts and Proportions

Questions from the assignment

1. Write an R program to create a stacked bar chart showing each brand's market share (%) by Quarter.
2. Modify the program to create a 100% (proportional) stacked bar chart.
3. Compare what each version communicates about the brands' relative performance.
4. Identify the brand with a consistently rising trend.
5. Explain a limitation of stacked bar charts when comparing individual segment sizes.

Aim

To visualize quarterly market shares for three brands in both stacked and proportional forms.

Procedure

Enter all twelve brand-quarter rows explicitly in long format, and order the Quarter and Brand factors. Use ggplot2::geom_col() for the stacked and proportional charts. ggplot2 functions are namespace-qualified, so no package attachment or repeated-value helper is needed.

Program

```
shares <- data.frame(
  Quarter = c("Q1", "Q1", "Q1", "Q2", "Q2", "Q2",
              "Q3", "Q3", "Q3", "Q4", "Q4", "Q4"),
  Brand = c("Brand P", "Brand Q", "Brand R",
            "Brand P", "Brand Q", "Brand R",
            "Brand P", "Brand Q", "Brand R",
            "Brand P", "Brand Q", "Brand R"),
  Share = c(35, 40, 25, 38, 36, 26,
            42, 33, 25, 45, 30, 25)
)
shares$Quarter <- factor(shares$Quarter,
                         levels = c("Q1", "Q2", "Q3", "Q4"))
shares$Brand <- factor(shares$Brand,
                       levels = c("Brand P", "Brand Q", "Brand R"))

ggplot2::ggplot(shares,
                ggplot2::aes(x = Quarter, y = Share, fill = Brand)) +
  ggplot2::geom_col(position = "stack") +
  ggplot2::labs(title = "Market Share by Quarter",
                x = "Quarter", y = "Market share (%)", fill = "Brand") +
  ggplot2::theme_minimal()
```

```
ggplot2::ggplot(shares,
                ggplot2::aes(x = Quarter, y = Share, fill = Brand)) +
  ggplot2::geom_col(position = "fill") +
  ggplot2::scale_y_continuous(
    labels = function(x) paste0(round(x * 100), "%")) +
  ggplot2::labs(title = "Proportional Market Share by Quarter",
                x = "Quarter", y = "Share of total", fill = "Brand") +
  ggplot2::theme_minimal()
```

Input:

RStudio-style practice mockup | Stacked bar charts and proportions

```
shares <- data.frame(
  Quarter = c("Q1", "Q1", "Q1", "Q2", "Q2", "Q2",
    "Q3", "Q3", "Q3", "Q4", "Q4", "Q4"),
  Brand = c("Brand P", "Brand Q", "Brand R", "Brand P",
    "Brand Q", "Brand R", "Brand P", "Brand Q", "Brand R",
    "Brand P", "Brand Q", "Brand R"),
  Share = c(35, 40, 25, 38, 36, 26, 42, 33, 25, 45, 30, 25)
)
ggplot2::ggplot(shares, ggplot2::aes(Quarter, Share, fill = Brand)) +
  ggplot2::geom_col(position = "stack")
ggplot2::geom_col(position = "fill")
```

Console / sample input (illustrative)

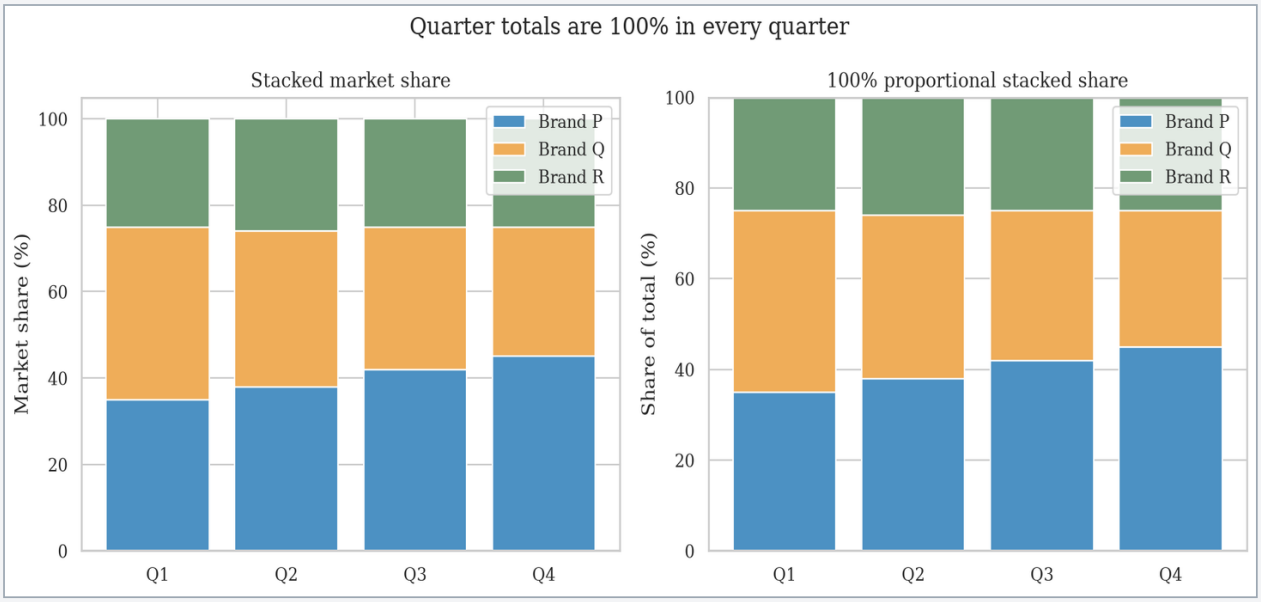
```
> Data and code are shown for illustration; this is not a live software capture.

> See the Program section for the complete script.
```

.

Output

RStudio-style practice mockup | Plots: stacked and proportional bars



Illustrative output from supplied data; not an actual RStudio screenshot.

Result

1. The first ggplot stacks each brand's percentage within each quarter. The second uses position = "fill" to scale each quarter to 100%.

2. For the proportional chart, position = "fill" converts each stacked total to a common height and the y-axis labels are shown as percentages.

3. A stacked chart shows segment values and their totals; a 100% stacked chart emphasizes each brand's share of the quarter total. Here both charts have the same relative proportions because all four quarter totals already equal 100%.

4. Brand P rises consistently, from 35% in Q1 to 45% in Q4. Brand Q declines, while Brand R stays close to 25%.

5. Only the bottom segment shares a common baseline. Comparing the middle and top segments across quarters is harder; a grouped bar chart or direct value labels may be clearer.

Question 5: Highlighting with Color

Questions from the assignment

1. Write a Python program to create a bar chart highlighting the expense category with the highest monthly spend.
2. Explain how highlighting improves data interpretation.
3. Identify the highest-spending category.
4. Discuss how excessive use of color can affect interpretation.
5. Suggest one alternative highlighting technique other than color.

Aim

To compare five monthly expense categories and use restrained color to make the largest expense immediately noticeable.

Procedure

Enter the category names and rupee amounts in two lists. Find the maximum with max(), assign one contrasting color to that bar and a quiet color to the others, then label each bar with its rupee amount. Matplotlib is required.

Program

```
import matplotlib.pyplot as plt

categories = ["Rent", "Groceries", "Travel", "Utilities", "Entertainment"]
spend = [15000, 8000, 4000, 3000, 2500]
highest = max(spend)
colors = ["tomato" if value == highest else "lightsteelblue"
          for value in spend]

bars = plt.bar(categories, spend, color=colors)
plt.title("Monthly Expenses: Highest Category Highlighted")
plt.ylabel("Monthly spend (INR)")
plt.ylim(0, highest + 3000)
plt.xticks(rotation=20, ha="right")
for bar in bars:
    value = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, value + 250,
             "Rs " + format(int(value), ","), ha="center")
plt.tight_layout()
plt.show()
```

Input

RStudio-style practice mockup | Highlighting with color (Python)

```python
import matplotlib.pyplot as plt
categories = ["Rent", "Groceries", "Travel",
              "Utilities", "Entertainment"]
spend = [15000, 8000, 4000, 3000, 2500]
highest = max(spend)
colors = ["tomato" if value == highest else "lightsteelblue"
          for value in spend]
bars = plt.bar(categories, spend, color=colors)
plt.title("Monthly expenses")
plt.ylabel("Monthly spend (INR)")
plt.show()
```

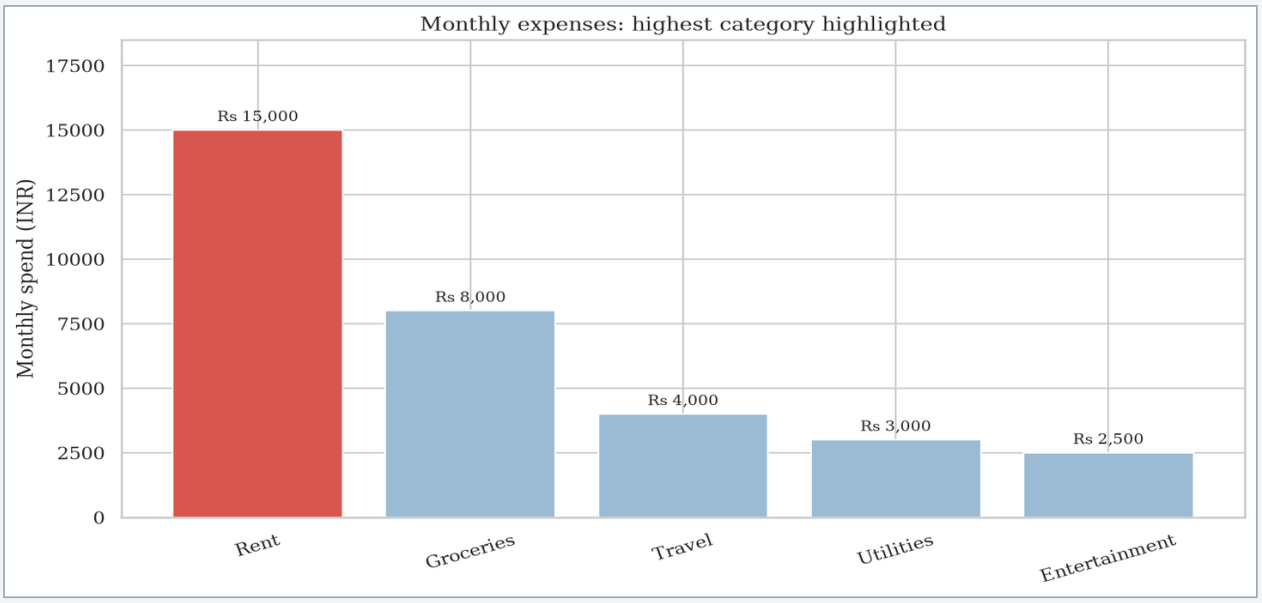
Console / sample input (illustrative)

> Data and code are shown for illustration; this is not a live software capture.

> See the Program section for the complete script.

Output

RStudio-style practice mockup | Plots: highlighted expense category



Illustrative output from supplied data; not an actual RStudio screenshot.

Result

1. The Python program draws a bar chart and highlights the maximum value with a contrasting color.

2. A single highlight directs attention to the main finding while leaving the other categories easy to compare against the same scale.

3. Rent is the highest-spending category at ₹15,000 per month.

4. If many bars use strong or unrelated colors, the highlight loses emphasis and viewers may infer categories that the colors do not actually represent. Use a limited, consistent palette.

5. Add a direct text annotation or outline around the Rent bar. Clear data labels can highlight the maximum without relying on color.